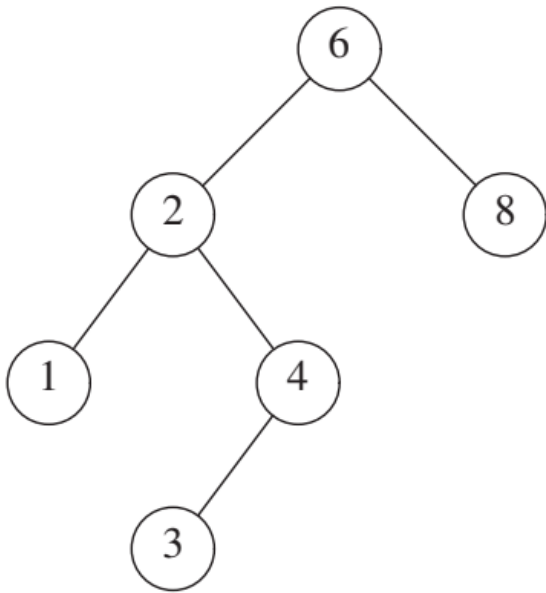


Example for Binary Search Tree: Insert

For example 5.1: if there is **the exist** BinarySearchTree's instance variable name **bst** contains the numbers [1, 2, 3, 4, 6, 8] (In-order traversal) as the image following:

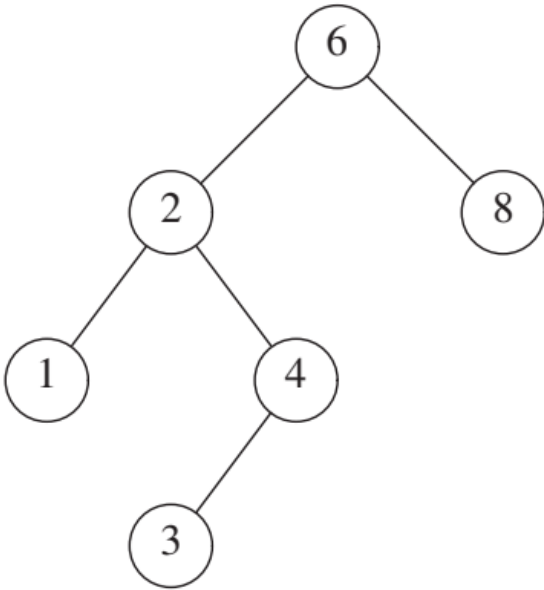


Write the steps for the `_insert_recursive()` function when insert 5 into the instance variable `bst`. In each round, consists of

- 1) what are the values of the parameters `value` and `current_node` of the `_insert_recursive()` function, and
- 2) what statements are executed under the if-else condition that is checked as true.

Each iteration must write 2 things.

```
in main program
bst = BinarySearchTree()
...
bst.insert(5)
```



Answer:

Each iteration for the `_insert_recursive()` function when insert 5

First round value = 5 , current_node = **6**
`_insert_recursive(current_node.left, value)`

round 2: value = 5 , current_node = **2**
`_insert_recursive(current_node.right, value)`

round 3: value = 5 , current_node = **4**
`current_node.right = Node(value)`



~~round 4: value = 5 , current_node = None~~
if current_node.right is None: **is True**
 current_node.right = Node(value)

[สาระสำคัญและเฉลยเหตุผล] ทำไมถึงขีดฆ่า Round 4 (std delete round 4):

* วิเคราะห์เจาะลึกทำไมถึงไม่มี Round 4 ในอัลกอริทึม _insert_recursive ของอาจารย์:

1. ในรอบที่ 3 ตัวแปร current_node คือ Node(4) และ value คือ 5
 2. เงื่อนไข value > current_node.value (5 > 4) เป็นจริง
 3. โค้ดของอาจารย์จะตรวจสอบ 'if current_node.right is None:' ก่อนทำการ Recursive
 4. เนื่องจากลูกขวาของโหนด 4 ยังว่างอยู่ (None) เงื่อนไขจึงเป็น True ทันที
 5. โปรแกรมสั่ง 'current_node.right = Node(value)' ผูกโหนด 5 เข้ากับโหนด 4 ทันที
- > จึงสิ้นสุดการทำงาน ไม่มีการเรียก Recursion รอบที่ 4 (current_node = None) เด็ดขาด!

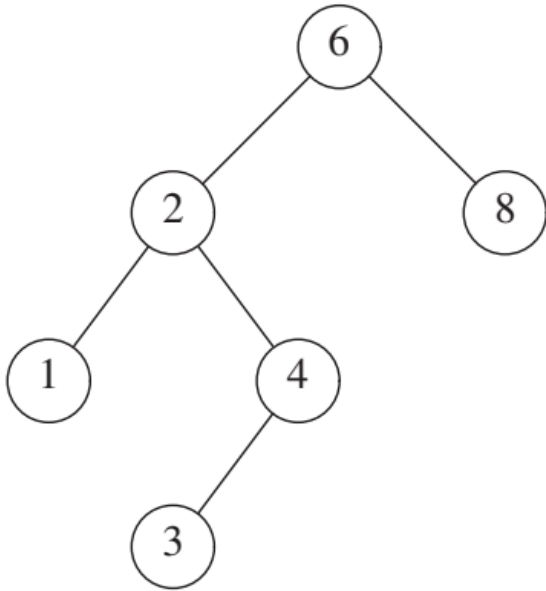
* คำเตือนข้อสอบ: หากนักศึกษาเขียน Round 4 จะถูกตัดคะแนนทันที เพราะการผูกโหนดเสร็จสิ้นตั้งแต่ Round 3

* โค้ดต้นฉบับจากสไลด์อาจารย์:

```
def _insert_recursive(self, current_node, value):  
    if value < current_node.value:  
        if current_node.left is None: current_node.left = Node(value)  
        else: self._insert_recursive(current_node.left, value)  
    elif value > current_node.value:  
        if current_node.right is None: current_node.right = Node(value)  
        else: self._insert_recursive(current_node.right, value)
```

Example for Binary Search Tree: Remove (4), one child.

For example 5.2: if there is **the exist** BinarySearchTree's instance variable name **bst** contains the numbers [1, 2, 3, 4, 6, 8] (In-order traversal) as the image following:

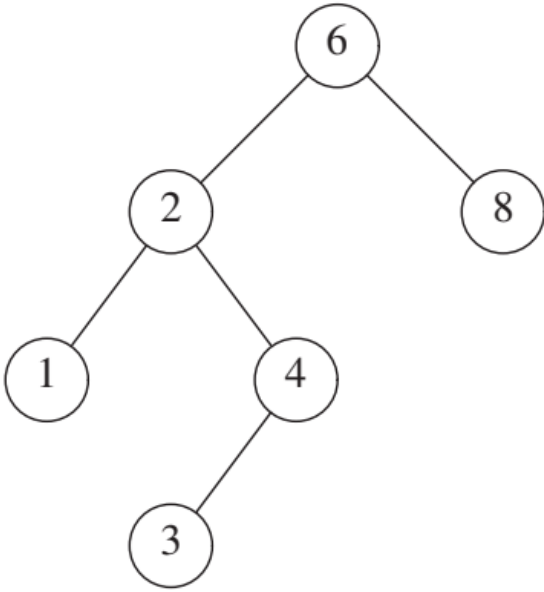


Write the steps for the `_delete_recursive()` function when remove 4 from the instance variable `bst`. In each round, consists of

- 1) what are the values of the parameters `value` and `current_node` of the `_delete_recursive()` function, and
- 2) what statements are executed under the if-else condition that is checked as true.

Each iteration must write 2 things.

Answer:



Each iteration for the `_delete_recursive()` function when remove 4

First round `value = 4` , `current_node = 6`
`current_node.left =`
`_delete_recursive(current_node.left, value)`

round 2: `value = 4` , `current_node = 2`
`current_node.right = _delete_recursive(current_node.right, value)`

round 3: `value = 4` , `current_node = 4`
`elif current_node.right is None: is True`
`return current_node.left`

[แสดงขั้นตอนการลบโหนดที่มี 1 ลูกอย่างเป็นทางการ]:

* สรุปขั้นตอนการลบโหนดที่มี 1 ลูก (One Child Delete - Node 4):

1. รอบที่ 1: `current_node = 6` -> `4 < 6` ไปกิ่งซ้าย (`current_node.left = _delete...`)
 2. รอบที่ 2: `current_node = 2` -> `4 > 2` ไปกิ่งขวา (`current_node.right = _delete...`)
 3. รอบที่ 3: `current_node = 4` -> เจอโหนดเป้าหมาย! โหนด 4 ไม่มีลูกขวา (`right is None`)
จึงส่งกลับลูกซ้าย (Node 3) กลับขึ้นไปผูกติดกับกิ่งขวาของโหนด 2 แทนที่โหนด 4
- > ผลลัพธ์: โหนด 4 ถูกลบสำเร็จ โหนด 3 ถูกเลื่อนขึ้นมาเป็นลูกขวาของโหนด 2 อย่างสมบูรณ์